

Relógio e alarme com FreeRTOS

ARDUINO MEGA 2560 · LCD 16x2

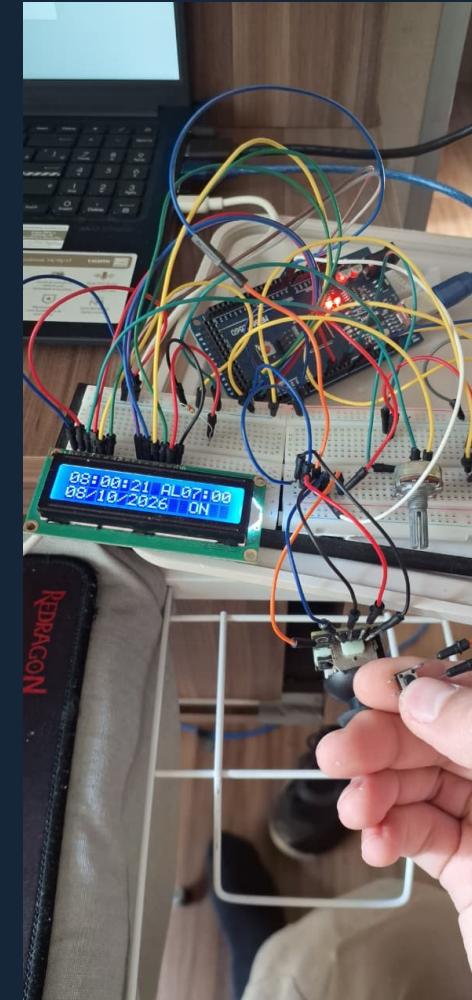
**Quatro tarefas.
Um único dono do estado.**

Operação local, comunicação por filas
e contagem de tempo em Timer1.

Pedro Lucas dos Reis Silva · RA 2272040

Engenharia de Computação · UTFPR Apucarana

Professor Adalberto Lazarini



O protótipo atende aos quatro requisitos

A configuração e a operação acontecem no próprio dispositivo.

01 Exibição local

Data e hora no LCD 16×2.

02 Ajuste por joystick

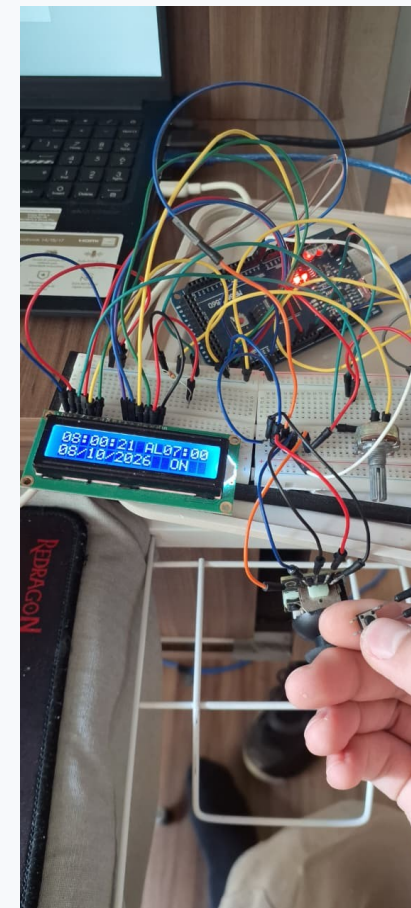
Hora, segundos, data e alarme.

03 Alarme

Mensagem no LCD, buzzer e LED.

04 Soneca

Duração configurável: 1 a 10 min.



Fonte: Suficiencia_2026_2.pdf · fotografias do autor, 08/10/2026

Por que usar RTOS neste relógio?

O RTOS organiza a concorrência; a máquina de estados organiza o comportamento.

Laço principal / bare metal

A aplicação define a sequência.

Menor custo de memória.

Também funciona com estados e ISRs.

FreeRTOS preemptivo

Prioridades e esperas pelo kernel.

Filas para trocar mensagens.

Custo: pilhas e troca de contexto.

Um núcleo: as tarefas alternam a CPU; não executam em paralelo.

O estado tem um único dono

As tarefas comunicam intenção e snapshots, sem compartilhar o calendário para escrita.



Saídas: caixas de 1 posição

Quem lê o botão não decide a regra. Quem escreve no LCD não altera a hora.

Fonte: RelogioFinal.ino · tarefaRelogio(), publicar(), enviar()

Prioridade define quem executa quando há disputa

A tarefa de maior prioridade também precisa liberar a CPU.

TAREFA	PRIOR.	RESPONSABILIDADE	COMO AGUARDA
Alarme	3	Buzzer e LED	Fila / fase sonora
Relógio	2	Calendário e regras	vTaskDelayUntil
Botões	2	Leitura e eventos	vTaskDelayUntil
Display	1	LCD e diagnóstico	Fila de snapshots

Bloqueada no kernel $\neq$ esperando em um laço que ocupa a CPU.

Mutex protege acesso; fila transporta informação

A evolução central foi retirar a escrita compartilhada do estado civil.

Antes: mutex

Adquirir → acessar → liberar.
Estrutura compartilhada.
Herança de prioridade no kernel.

Agora: filas

Evento → controlador → cópia.
Estado privado do relógio.
LCD sem lock compartilhado.

Entrada: FIFO de 16 eventos
Preserva a ordem dos comandos aceitos.

Saídas: xQueueOverwrite
Conserva o estado mais recente.

Fontes: versões preservadas · queue.h / queue.c · documentação de mutexes do FreeRTOS

Tick do RTOS e relógio civil são bases distintas

Timer0, Timer1 e Timer2 têm funções diferentes nesta implementação.

Watchdog → escalonador

Tick nominal $\approx$ 16 ms.

Agenda esperas e tarefas.

Timer1 → calendário

CTC a 100 Hz.

Uma contagem nominal a cada 10 ms.

Timer0 → millis()

Base do core Arduino.

Diagnóstico; não é o RTC do projeto.

Timer2 → tone()

Geração da onda de áudio.

Buzzer a 2 kHz durante a fase ativa.

A ISR registra tempo; a tarefa atualiza o calendário

Prescaler = 64 · OCR1A = 2499 · clock nominal = 16 MHz

$$16.000.000 / [64 \times (2499 + 1)] = 100 \text{ Hz}$$

```
ISR(TIMER1_COMPA_vect) {  
    ++ticks100Hz;  
}  
  
ATOMIC_BLOCK(ATOMIC_RESTORESTATE) {  
    copia = ticks100Hz;  
}
```

Leitura íntegra no AVR

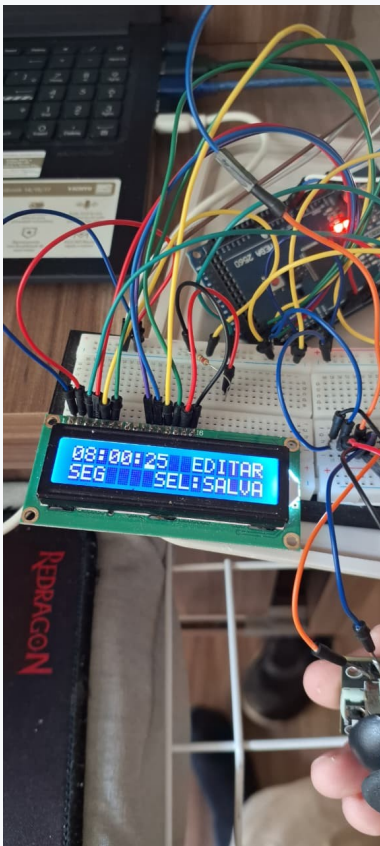
Contador de 32 bits em CPU de 8 bits.
volatile não basta: leitura atômica.
LCD fora da seção crítica.

agora – anterior → contagens decorridas → calendário e prazos

Trechos de RelogioFinal.ino · lerTempo() e tarefaRelogio()

Editar o alarme não interrompe o relógio

A edição usa um rascunho; somente o controlador aplica a configuração.



Navegação

← / → escolhe o campo

↑ / ↓ altera o valor

SEL salva · D5 cancela a edição

Debounce: 50 ms de estabilidade.

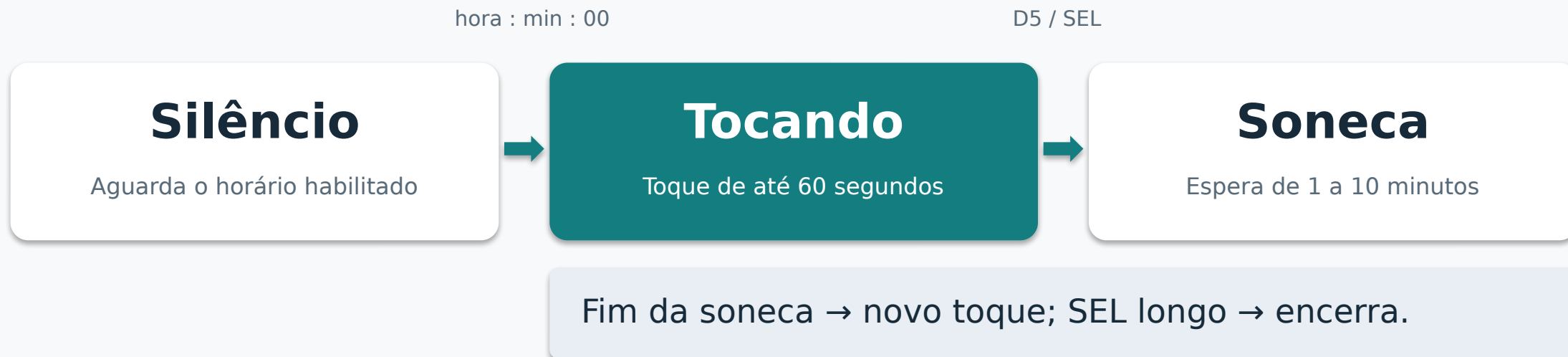
SEL longo: um evento após 1 s.

Sem esperar o botão ser solto em um laço.

Fonte: RelogioTipos.h · Botao::ler(), Eixo::ler() · comandar()

A soneca usa duração relativa, não hora do dia

O mesmo evento de soneca pode vir de D5 ou do clique em SEL.



23:59:50 + 60 segundos → 00:00:50

A mudança de dia não invalida o prazo restante.

Fonte: comandar(), consumirPrazo(), avaliar() · exemplo de meia-noite exercitado nos testes

Precisão física não é a mesma coisa que prioridade

Três fenômenos precisam ser separados na análise do horário.

Acerto do horário

Diferença de horário
no instante em que
o relógio é acertado.

Deriva

Erro que cresce
com a frequência real
do oscilador.

Latência

Tempo até a tarefa
processar um evento
ou atualizar a saída.

**Timer1 separa o calendário do watchdog.
A exatidão continua dependendo da referência física.**

O comportamento foi verificado em etapas

Montagem funcional, núcleo testado e firmware compilado para o Mega.

13

grupos de testes
do núcleo aprovados

Calendário e prazos

Meia-noite e ano bissexto.
Soneca e fim do toque.
Rollover e recuperação.

Entrada e display

Edição de segundos.
Debounce e clique longo.
Limites do LCD 16×2.

18.388 bytes de programa • 729 bytes de SRAM estática

Pilhas e filas acrescentam alocação em execução; o ensaio temporal do núcleo usa contagens nominais.

Demonstração: configurar, disparar e adiar

O projeto integra tarefas, eventos e interrupções em uma interface autônoma.

1

Configurar pelo joystick

Hora, segundos, alarme e soneca.

2

Observar o disparo

Mensagem no LCD, LED e buzzer.

3

Acionar D5 e encerrar

Soneca no LCD; SEL longo encerra o aviso.



Timer1: valores e significado dos registradores

CTC a 100 Hz · divisão do clock principal de 16 MHz.

TCCR1A = 0	Desconecta as saídas OC1 do controle do timer.
TCCR1B: WGM12	Seleciona CTC.
TCCR1B: CS11 CS10	Seleciona prescaler 64.
OCR1A = 2499	Comparação nominal a 100 Hz.
TIFR1: OCF1A = 1	Limpa uma comparação pendente.
TIMSK1: OCIE1A = 1	Habilita a interrupção de comparação A.

Trocar a divisão para 1 Hz não corrige o erro do mesmo oscilador.

Fonte: `iniciarTempo()` · `RelogioFinal.ino`

Quatro respostas importantes para a arguição

As garantias pertencem à arquitetura completa, não a uma API isolada.

“Fila elimina qualquer corrida?”

Não. Dono único + cópia + leitura atômica sustentam a proteção.

“Nunca há bloqueio?”

Há espera pelo kernel.
Não há espera ocupada nos botões.

“E se a entrada ficar cheia?”

Envio sem espera; perda notificada e mensagem para repetir o comando.

“E a memória das tarefas?”

1.568 bytes nas quatro pilhas.
Margem observada por high-water mark.

Fontes e material da entrega

A teoria acompanha o código da biblioteca e o firmware efetivamente entregue.

Enunciado do exame

Adalberto Lazarini · UTFPR Apucarana · 2026.2

Arduino FreeRTOS 11.1.0-3

tasks.c · queue.c · queue.h · FreeRTOSVariant.h

Arduino AVR Core 1.8.8

wiring.c · Tone.cpp

AVR-LibC e Microchip

ATOMIC_BLOCK e configuração de interrupções megaAVR

Relatório, firmware e fotografias

github.com/plreis/Sistemas-Embarcados